Numerical Problem Solving

Solving $Ax = b$ Systems Efficiently

Kacper Cichy
Gdańsk University of Technology – Data Analytics

August 2026

1 Objective

The primary goal of this session is to achieve fluency in using numerical solvers (specifically `scipy.linalg.solve`) for linear systems, rather than explicitly computing the inverse matrix A^{-1}.

2 Theoretical Background: The Problem with A^{-1}

In linear algebra, the system of linear equations is defined as:

$$Ax = b$$

Mathematically, to find the vector x, one would multiply both sides by the inverse of A:

$$x = A^{-1}b$$

However, in **computational mathematics and data analytics**, calculating A^{-1} explicitly is considered a poor practice for two main reasons:

1. **Computational Complexity:** Calculating the inverse of an $N \times N$ matrix requires $O(N^3)$ operations. For large matrices, this becomes a severe performance bottleneck.
2. **Numerical Instability:** Floating-point arithmetic introduces rounding errors. Calculating A^{-1} and then multiplying it by b magnifies these errors, especially if matrix A is ill-conditioned (i.e., its condition number is high).

3 The Solution: Solvers and LAPACK/BLAS

Instead of finding the inverse, modern numerical libraries solve the system directly using factorization methods (like **LU Decomposition**, which breaks A into lower and upper triangular matrices: $A = LU$).

When using `scipy.linalg.solve`, Python does not compute this naively. Under the hood, it delegates the operation to **LAPACK** (Linear Algebra PACKage) and **BLAS** (Basic Linear Algebra Subprograms) — highly optimized, hardware-level routines written in Fortran/C that utilize vectorization and optimal memory cache management.

4 Practical Test: 1000×1000 Matrix

The task is to find the vector x from a random 1000×1000 matrix without using `inv()`. Below is the Python implementation that compares the performance of both approaches.

```
import numpy as np
import scipy.linalg as la
import time

# 1. Generate a random 1000x1000 matrix A and a vector b of size 1000
N = 1000
A = np.random.rand(N, N)
b = np.random.rand(N)

# --- APPROACH 1: The Bad Way (Matrix Inversion) ---
start_time = time.perf_counter()
A_inv = la.inv(A)
x_bad = np.matmul(A_inv, b)
time_bad = time.perf_counter() - start_time
print(f"Time using inv(): {time_bad:.5f} seconds")

# --- APPROACH 2: The Pro Way (SciPy Solver) ---
start_time = time.perf_counter()
x_good = la.solve(A, b)
time_good = time.perf_counter() - start_time
print(f"Time using solve(): {time_good:.5f} seconds")

# Verification: Check if both methods yield the same numerical result
# using numpy.allclose which accounts for floating-point tolerance
is_same = np.allclose(x_bad, x_good)
print(f"Are results numerically identical? {is_same}")

```

5 Conclusion

By running the code above, the `scipy.linalg.solve` method will consistently execute faster and utilize less memory. In advanced SciML and engineering contexts, explicitly calling `inv()` should be strictly avoided unless the inverse matrix itself is the required final output for theoretical analysis.